AWS Level 3 iteration 1 terminaldokumentation

Claudio Verdi | APL | 23 oktober 2026

Jag skapade en Ubuntu-server i AWS med Terraform, installerade Nginx via SSH och publicerade en enkel webbplats. Sidan verifierades lokalt på servern, från min dator och i webbläsaren. Efter testerna tog jag bort instansen med Terraform och verifierade status terminated med AWS CLI.

Miljö och resurser

Egenskap	Värde
Region	eu-north-1
IAM	tabish-admin
Terraform / AWS provider	1.16.3 / hashicorp/aws 6.70.0
Instance ID	i-014006d7d2192e50c
Instanstyp	t3.micro
AMI	ami-06fb1802e2b757f09
OS	Ubuntu 24.04.5 LTS x86_64
VPC	vpc-00dcc7fb53fe9926b
Subnet	subnet-05d4dca4820d2eb94 / eu-north-1a
Security group	sg-0fb87d0f08a213466
Nyckelpar	devops.school.level3.tabish-admin
Tidigare publik IP	16.170.140.221
Privat IP	172.31.31.13
Nginx / Git	1.24.0 / 2.43.0

Referens: Level3 Iteration1 (1)(2)(1).sh. Arbetsgången genomfördes stegvis med anpassningar till den faktiska miljön.

Förberedelser och AWS identitet

Jag kontrollerade verktygen i Ubuntu under WSL. Terraform 1.16.3 och AWS CLI fanns installerade. jq saknades och installerades med apt. STS bekräftade åtkomst som IAM-användaren tabish-admin.

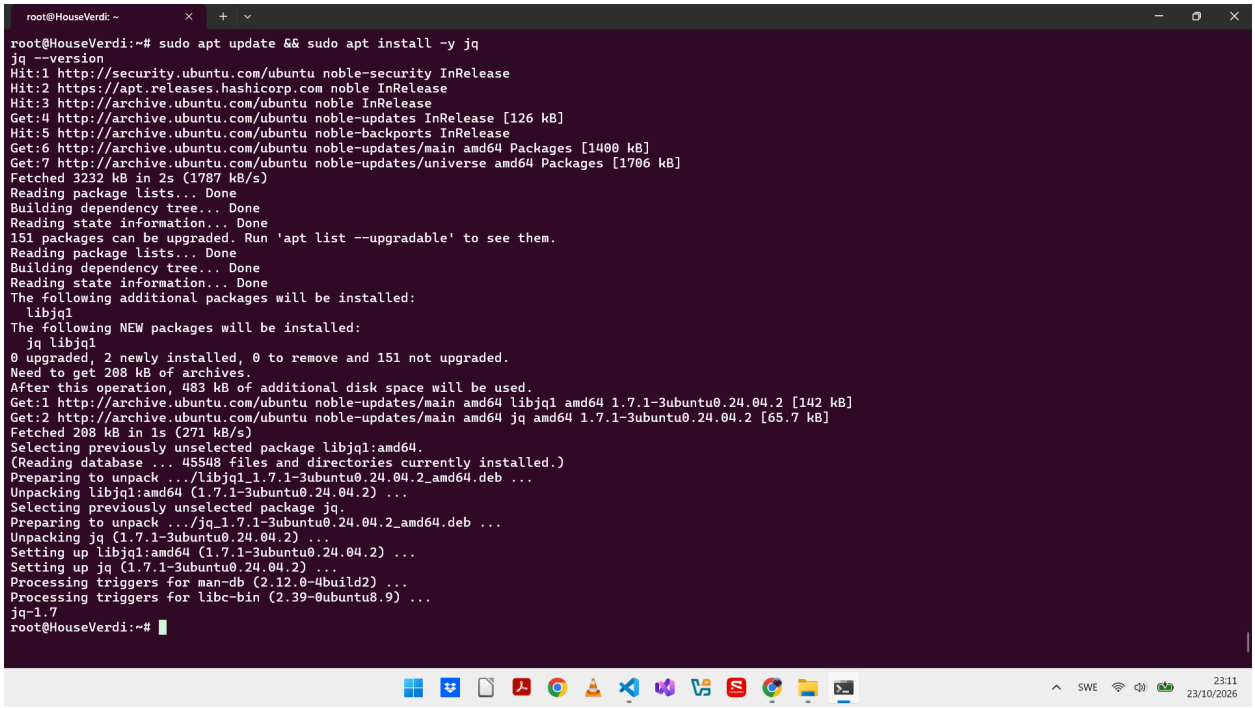


Bild 1. Installation av jq slutförd

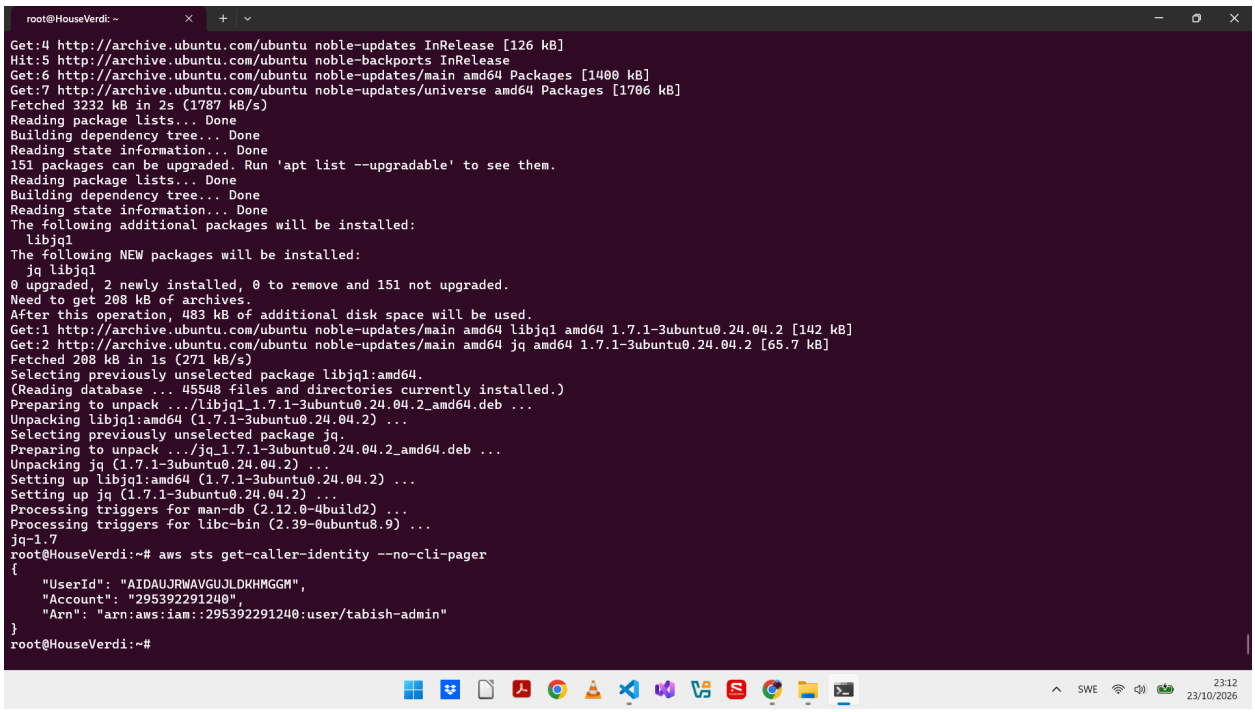


Bild 2. AWS identitet verifierad med STS

SSH nyckel och nätverk

Jag återanvände den befintliga lokala nyckeln ~/.ssh/id_rsa_level3 och importerade dess publika del till AWS. Den privata filen hade behörighet 600 och stannade på min dator. Jag använde standard-VPC:n och valde subnätet i eu-north-1a.

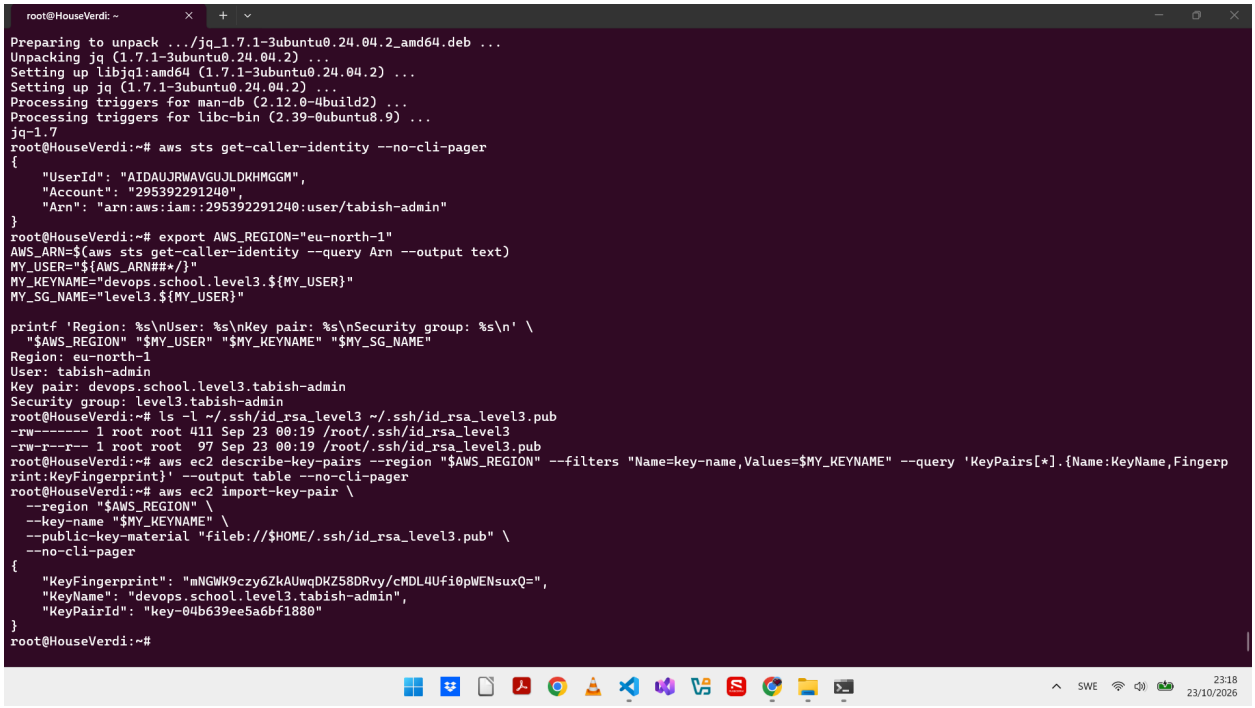


Bild 3. Publik nyckel importerad till AWS

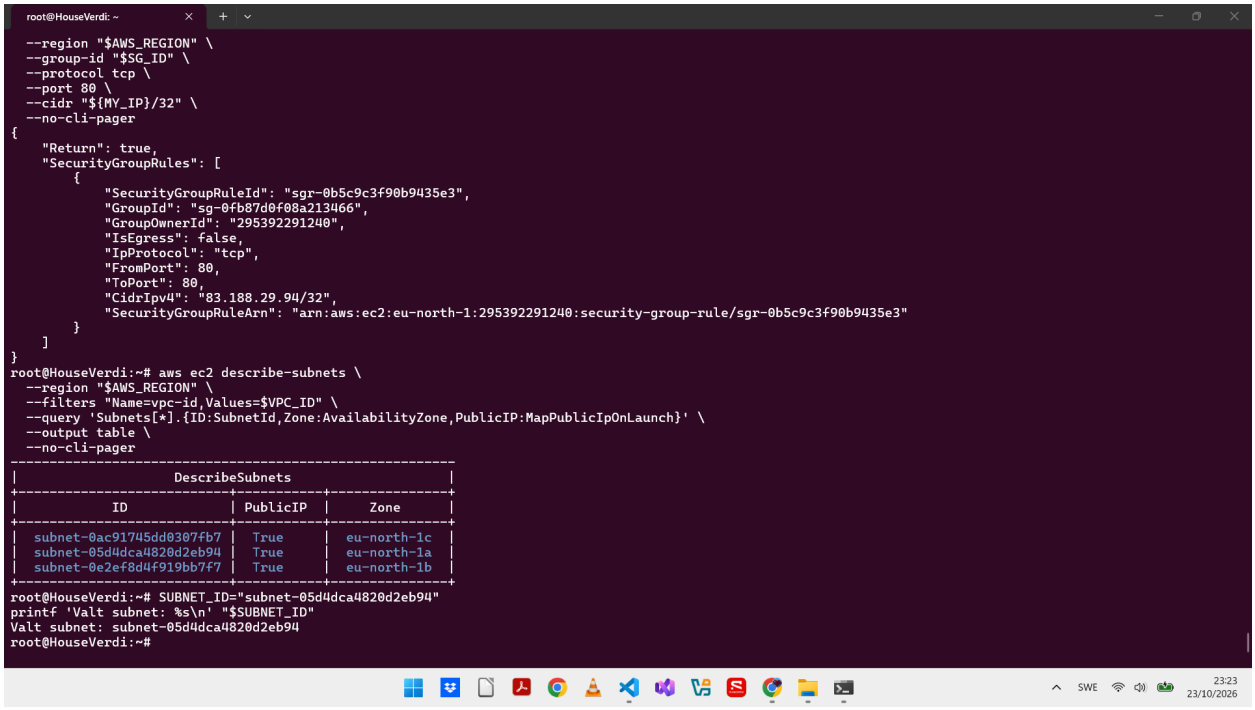


Bild 4. Valt subnät i eu-north-1a

Säkerhetsgrupp och Ubuntu avbildning

Jag skapade säkerhetsgruppen level3.tabish-admin och tillät TCP 22 och 80 från min dåvarande publika IP 83.188.29.94/32. Reglerna begränsade både SSH och HTTP till den adressen. Ubuntu-avbildningen hämtades med AWS CLI från Canonicals ägar-ID 099720109477.

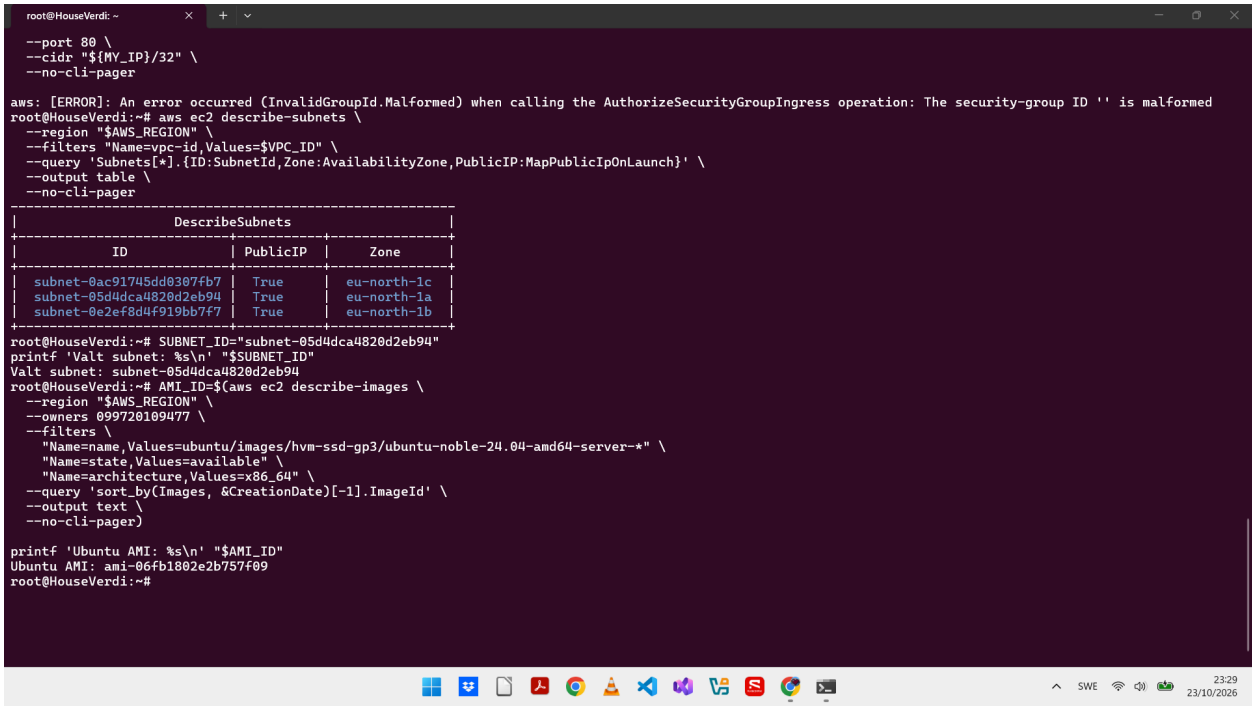


Bild 5. AMI-resultat och misslyckad upprepning med tom SG_ID

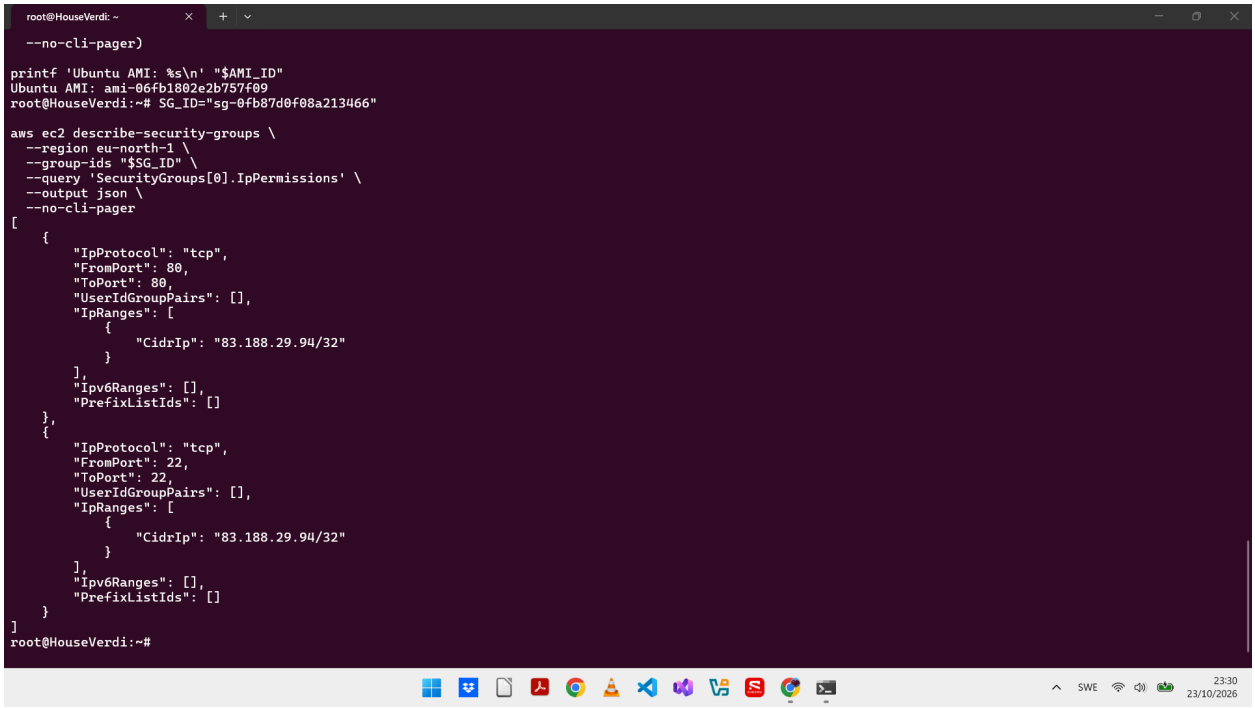


Bild 6. Återställd SG_ID och verifierade regler för port 22 och 80

Terraform konfiguration och initiering

För att bevara äldre filer skapade jag katalogen ~/aws-devops-level3-iteration1. Konfigurationen delades i providers.tf, variables.tf, main.tf, outputs.tf och terraform.tfvars. De fullständiga filerna finns i bilagan.

Terraform hanterade en EC2-instans. VPC och subnät fanns redan; nyckelparet och säkerhetsgruppen förbereddes med AWS CLI. Instansen fick publik IP, krav på IMDSv2 och krypterad rotdisk.

```
terraform fmt
terraform init
terraform validate
```

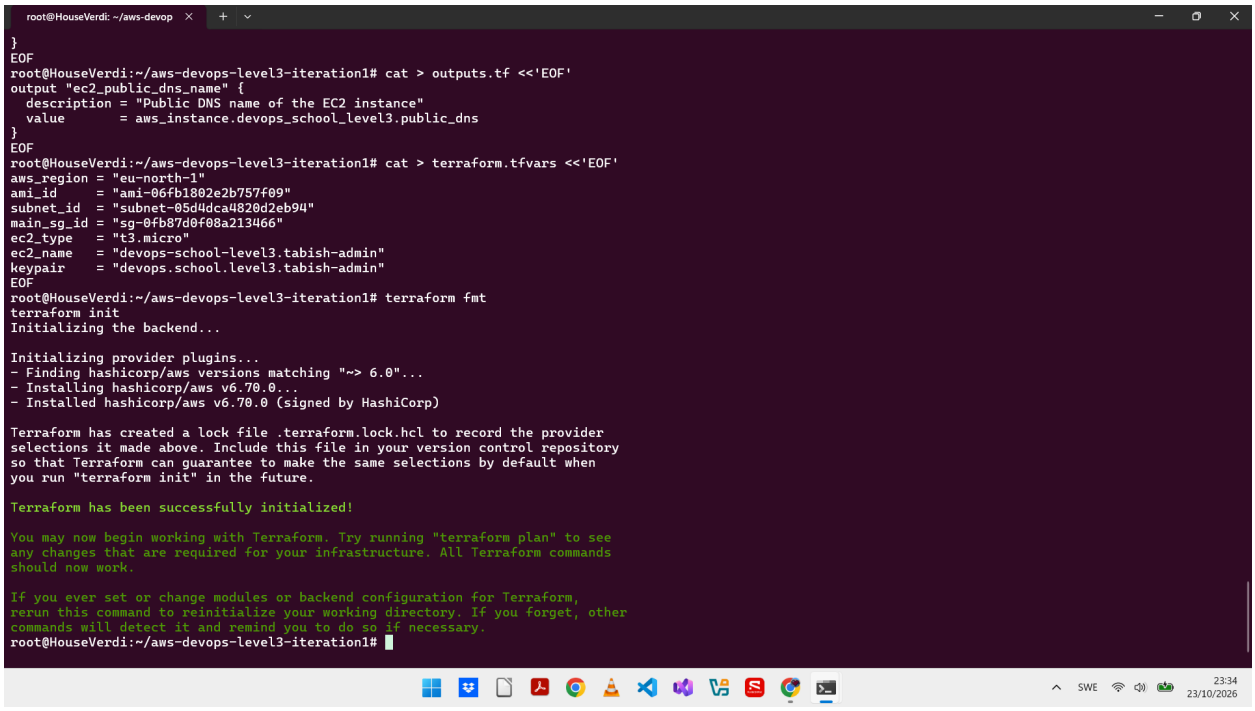


Bild 7. Initiering installerade AWS provider 6.70.0 och skapade låsfil

terraform validate gav Success! The configuration is valid. Därefter sparade jag planen med terraform plan -out=tfplan.

Planering och skapande av instansen

Planen visade 1 att skapa, 0 att ändra och 0 att ta bort. Jag tillämpade den sparade planen med terraform apply tfplan. Instansen skapades och Terraform lämnade ut dess publika DNS-namn.

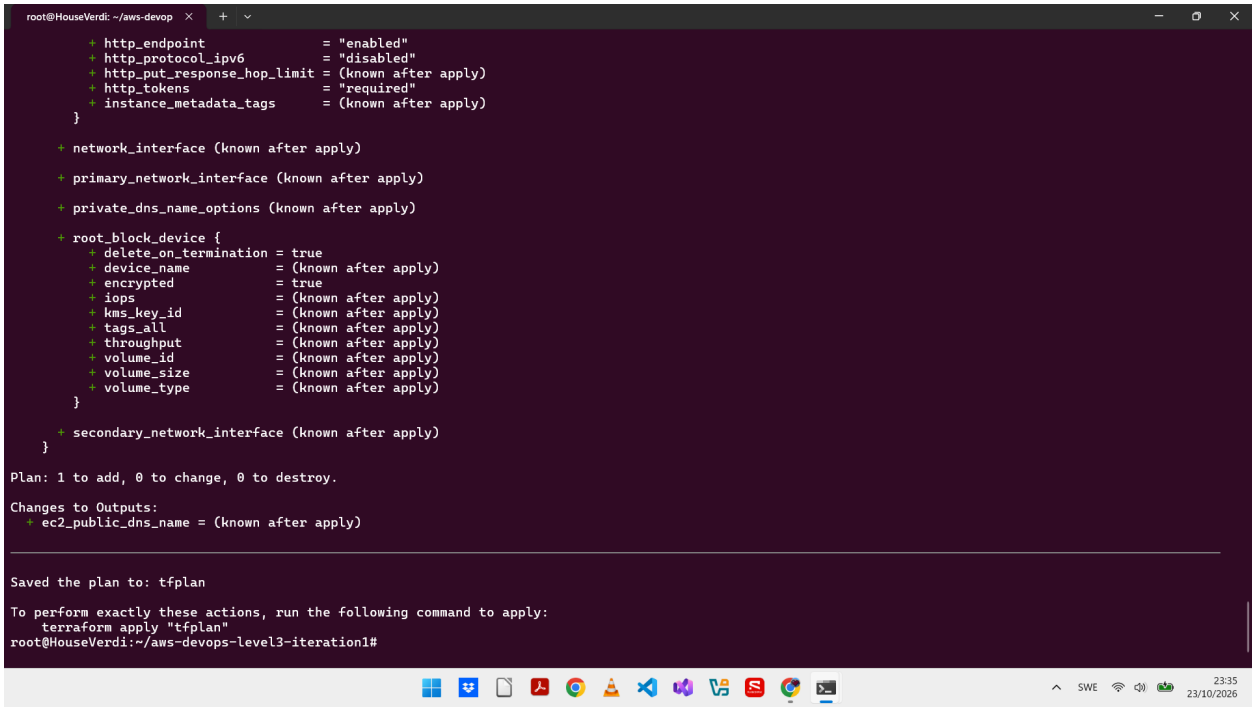


Bild 8. Sparad plan med en ny resurs

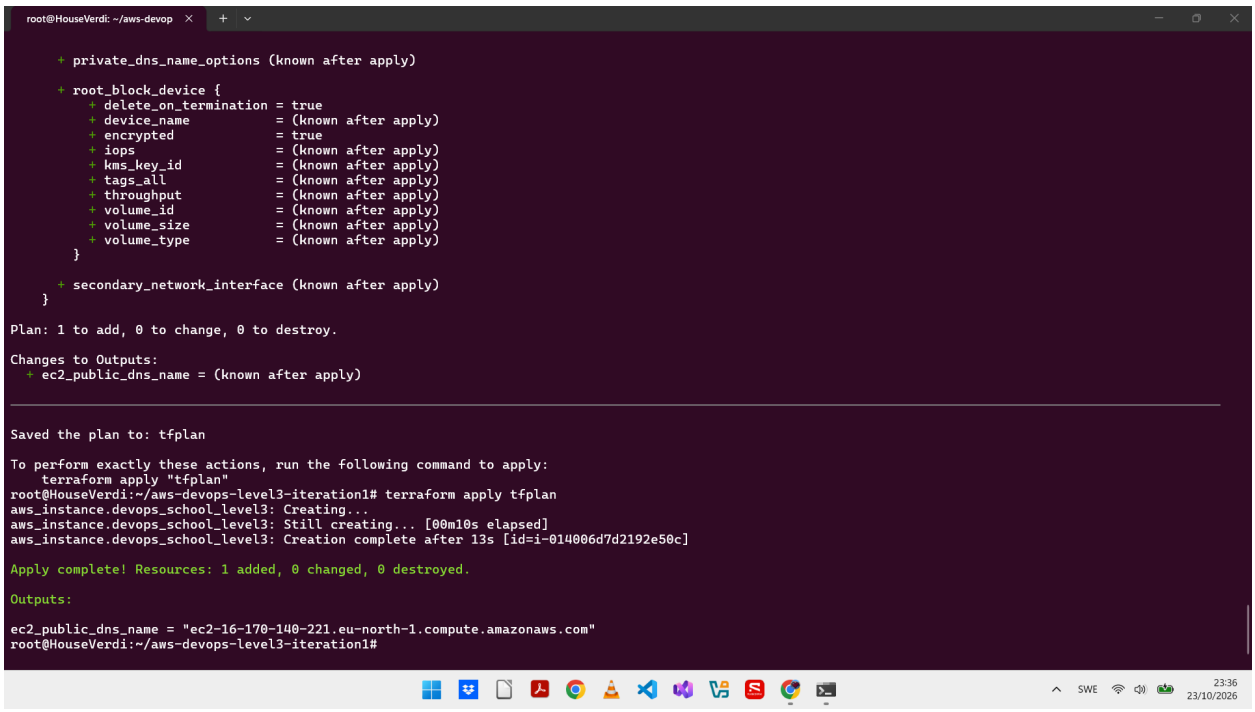


Bild 9. Lyckad apply med instans-ID och publikt DNS-namn

SSH anslutning och systemuppdatering

Jag hämtade DNS-namnet från Terraform och anslöt som ubuntu med den lokala privata nyckeln. Serverns inloggningsbanner visade Ubuntu 24.04.5 LTS. Uppdateringskommandot slutfördes med 0 uppgraderade paket.

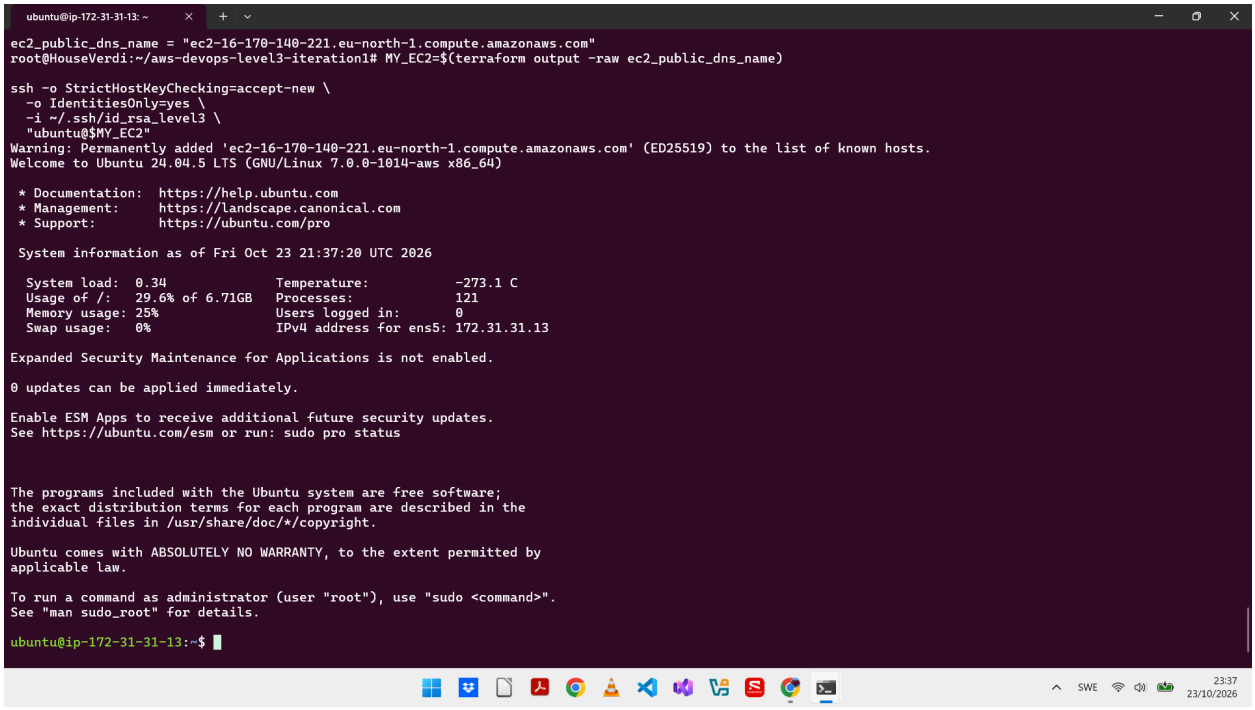


Bild 10. Lyckad SSH-inloggning på Ubuntu

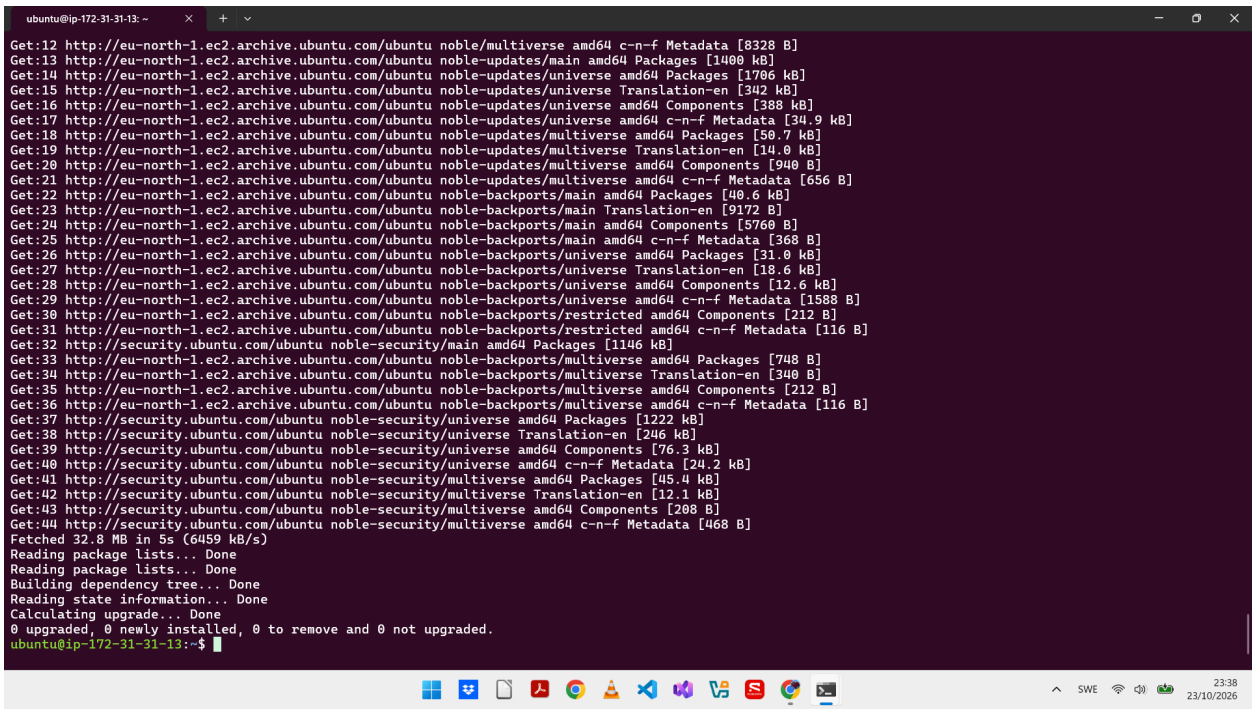


Bild 11. Systemets paketlistor och uppgraderingar kontrollerade

Installation av Nginx och hämtning av webbplatsen

Jag installerade Nginx och Git med apt. Nginx aktiverades med systemctl enable --now nginx; kontrollerna gav active och enabled. Därefter klonade jag webbplatsens filer och granskade katalogen.

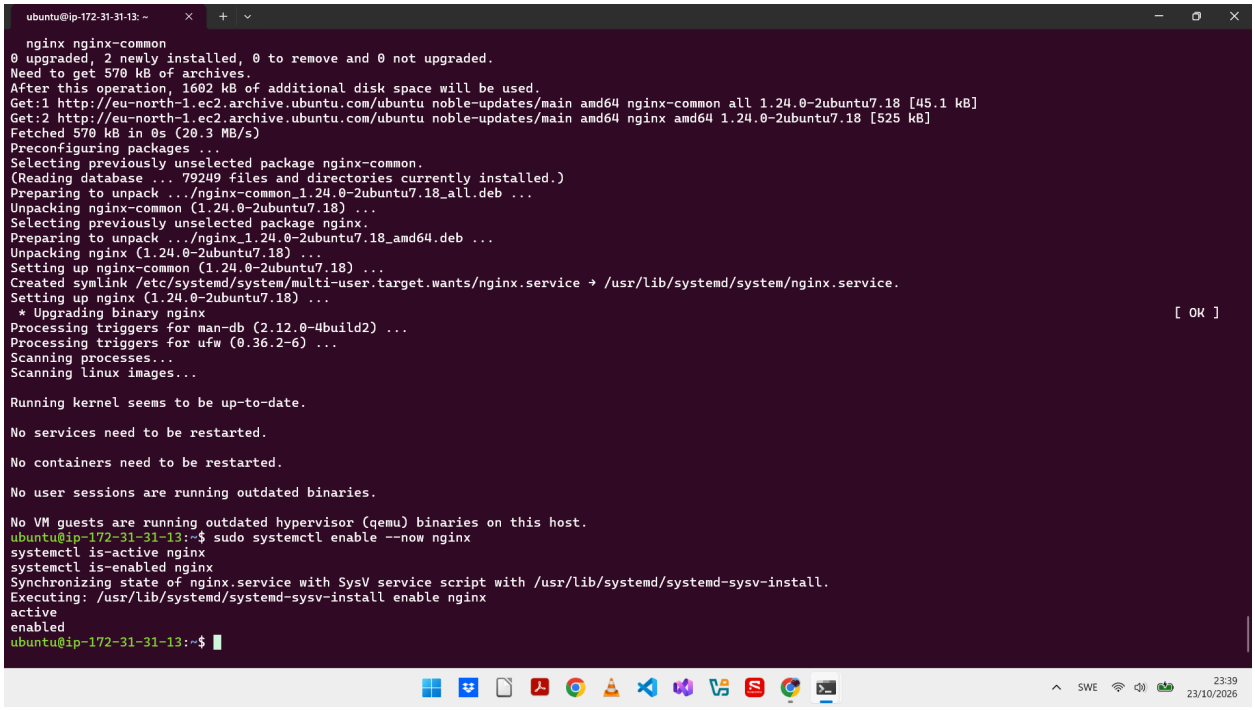


Bild 12. Nginx är aktiv och aktiverad för automatisk start

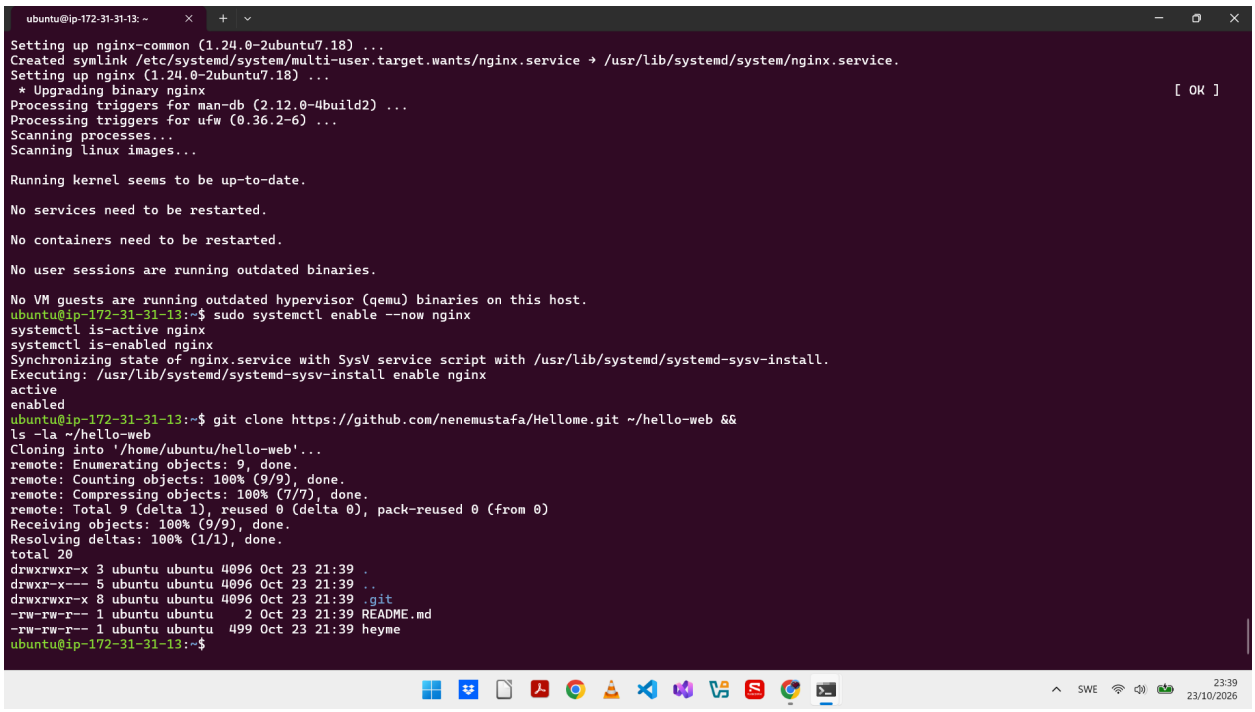


Bild 13. Kodförrådet innehöll filen heyme på 499 byte

Publicering och lokalt HTTP test

Guidens förväntade html-katalog saknades. Webbplatsens HTML låg i filen heyme, som jag kopierade till /var/www/html/index.html. nginx -t lyckades och jag laddade om tjänsten. curl mot localhost gav HTTP 200 och sidans HTML med rubriken Hey me!.

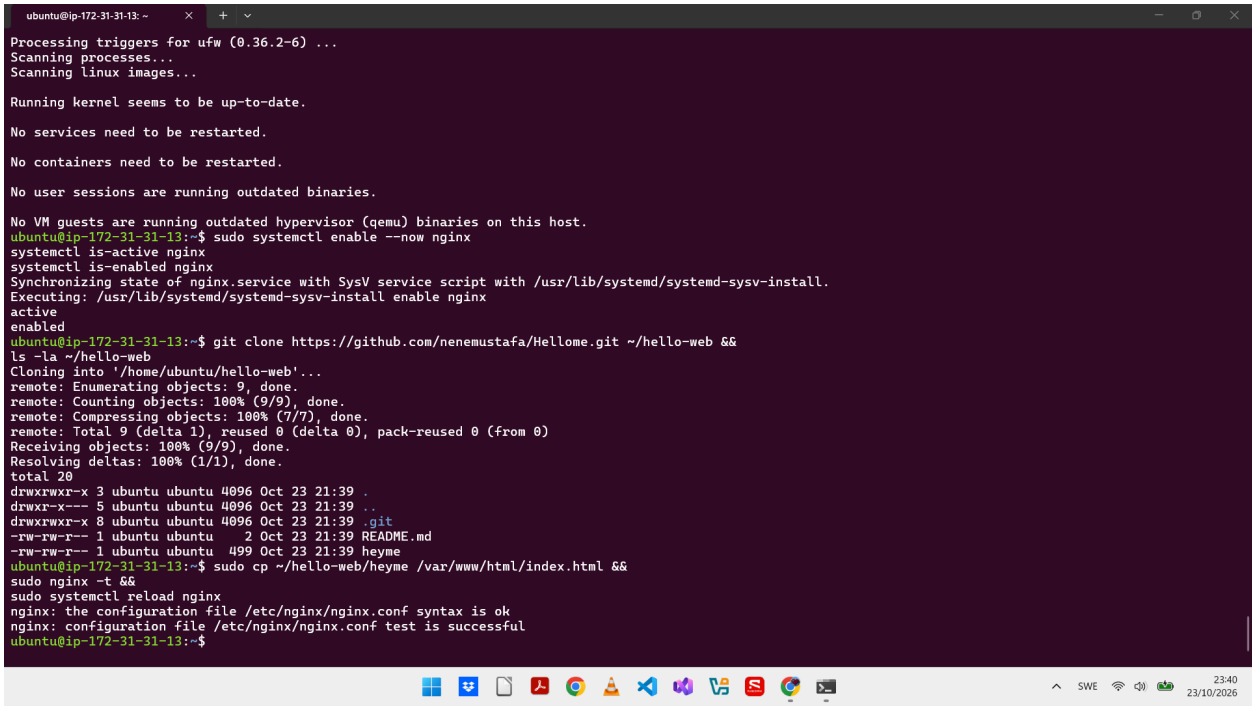


Bild 14. Kopiering till index.html och lyckat konfigurationstest

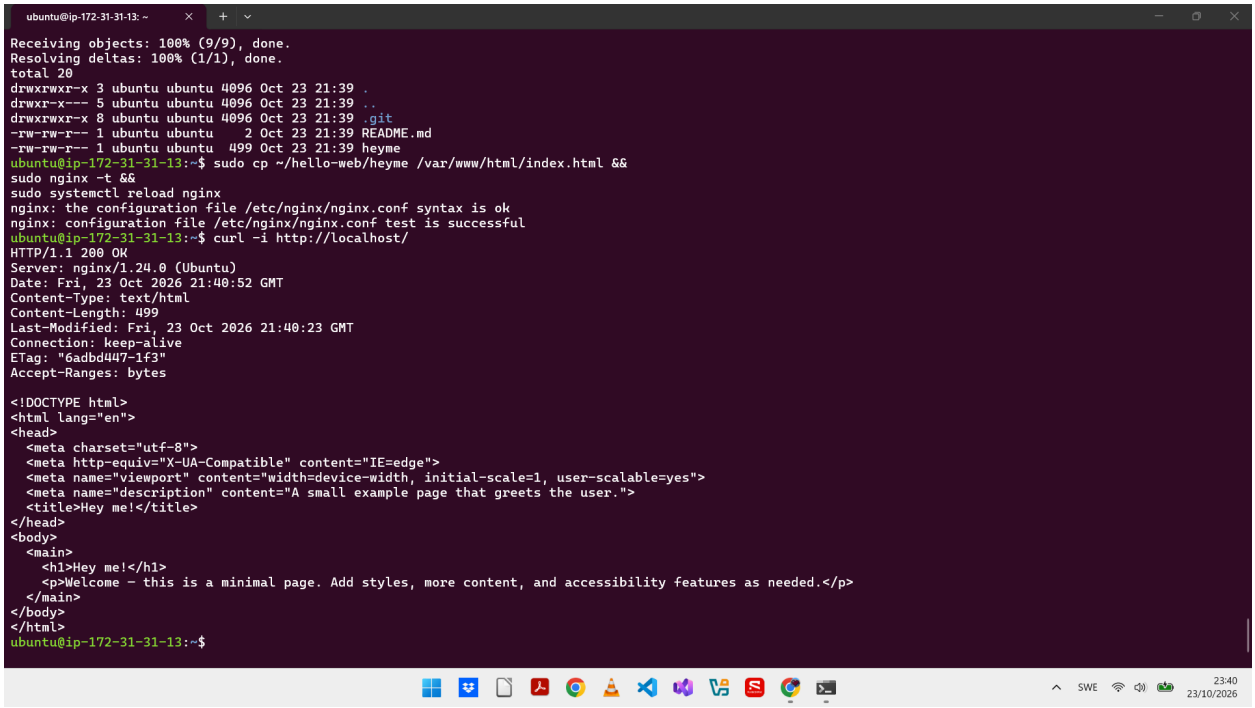


Bild 15. HTTP 200 från localhost med korrekt HTML

Webbläsare och externt HTTP test

Webbläsaren visade sidan på http://16.170.140.221/. Efter att jag lämnat SSH-sessionen körde jag curl från min lokala WSL-terminal mot instansens DNS-namn. Även det testet gav HTTP 200 och innehållet Hey me!.

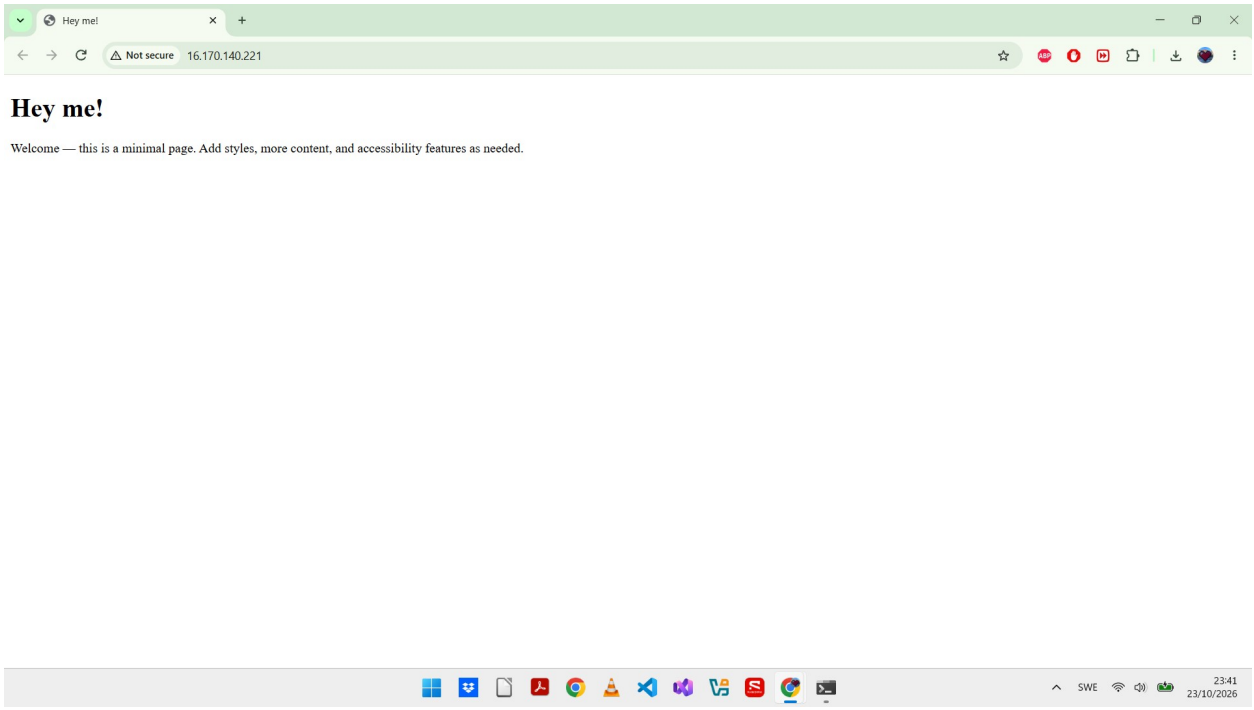


Bild 16. Webbplatsen på instansens publika IP

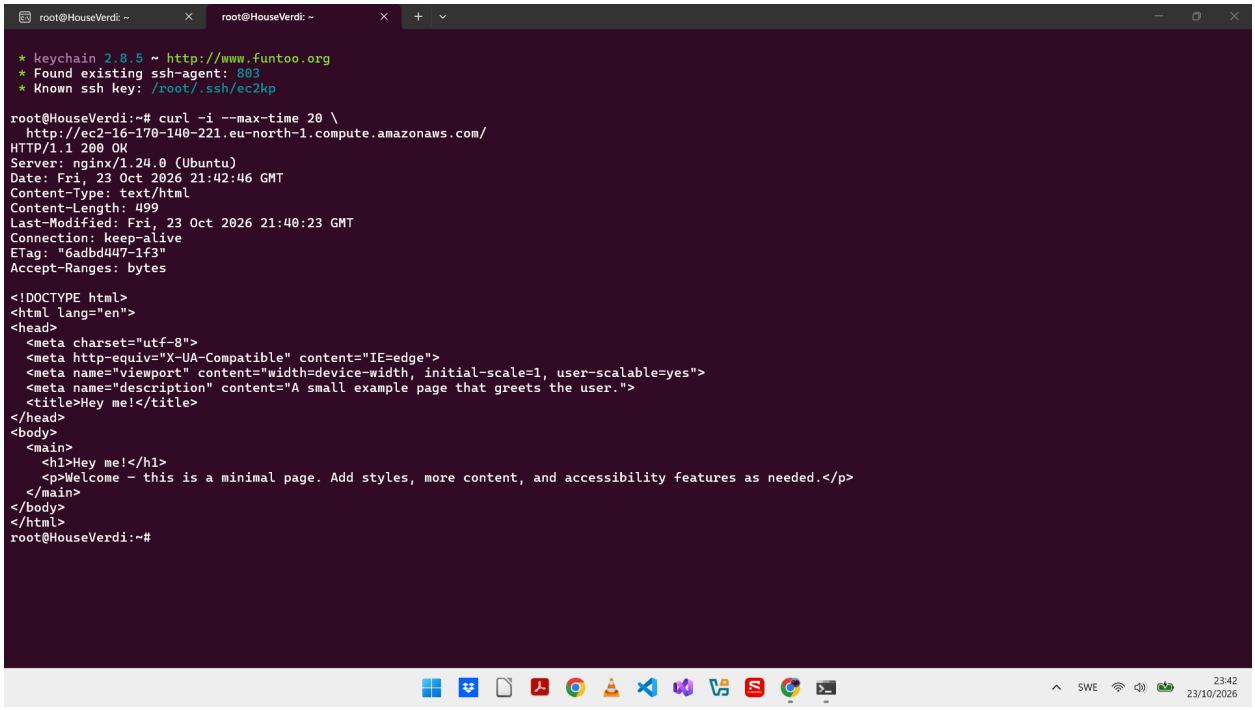


Bild 17. HTTP 200 och rätt HTML från den lokala datorn

Kontroll av Terraform och borttagning

Efter publiceringen körde jag terraform plan igen. Resultatet No changes visade att infrastrukturen stämde med konfigurationen. Därefter sparade jag en separat borttagningsplan med terraform plan -destroy -out=destroyplan.

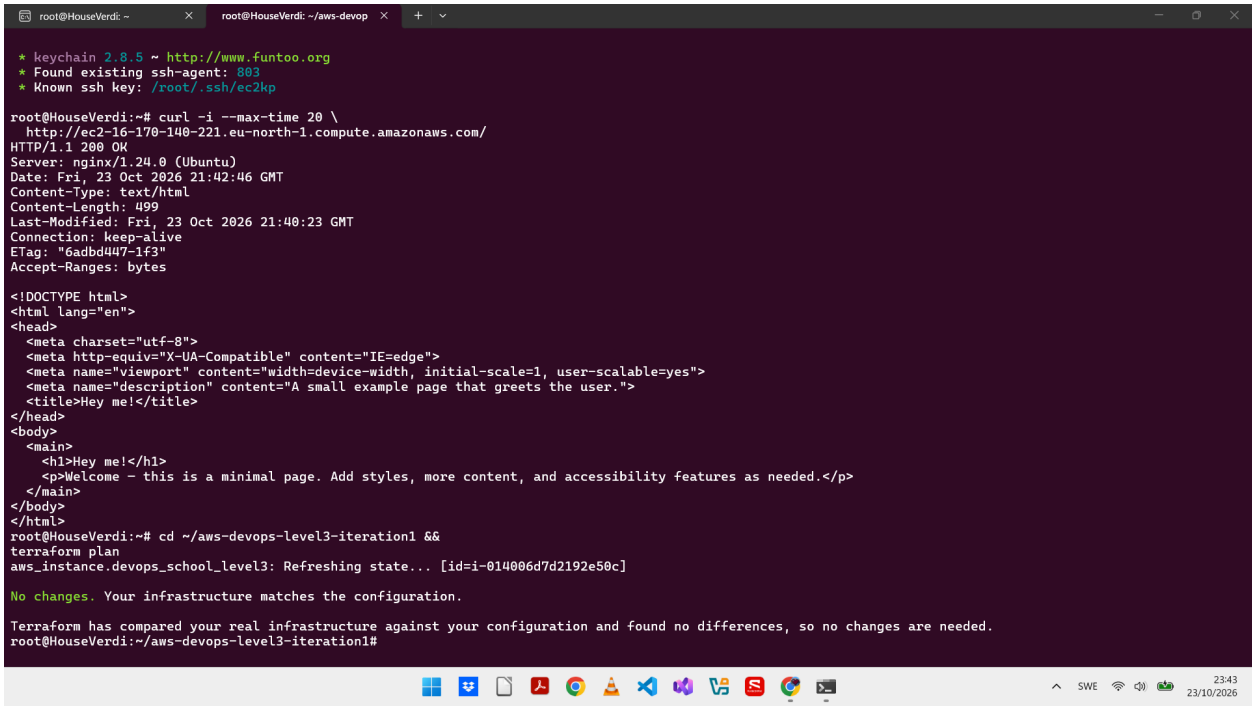


Bild 18. Terraform visar No changes efter publicering

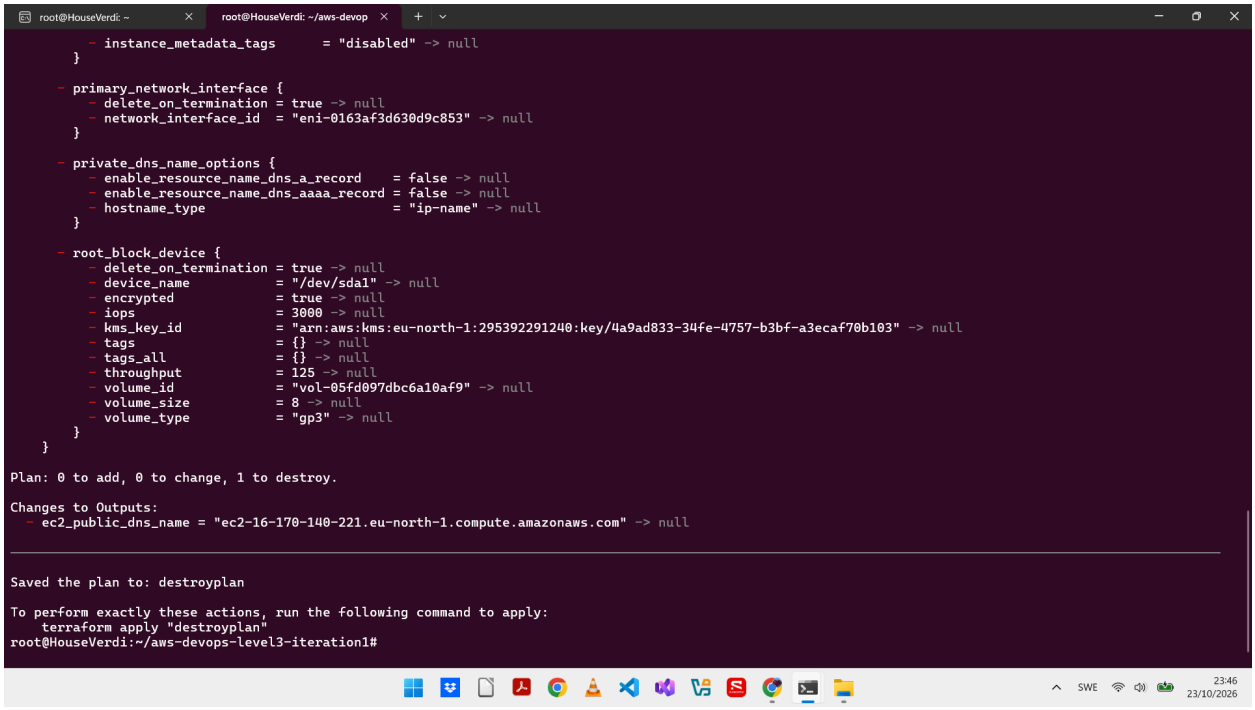


Bild 19. Borttagningsplan med en resurs och krypterad rotdisk

Verifierat avslut

Jag körde terraform apply destroyplan. Terraform rapporterade 0 added, 0 changed, 1 destroyed. En separat AWS CLI-kontroll av i-014006d7d2192e50c gav terminated den 23 oktober 2026 kl. 23.49 svensk tid.

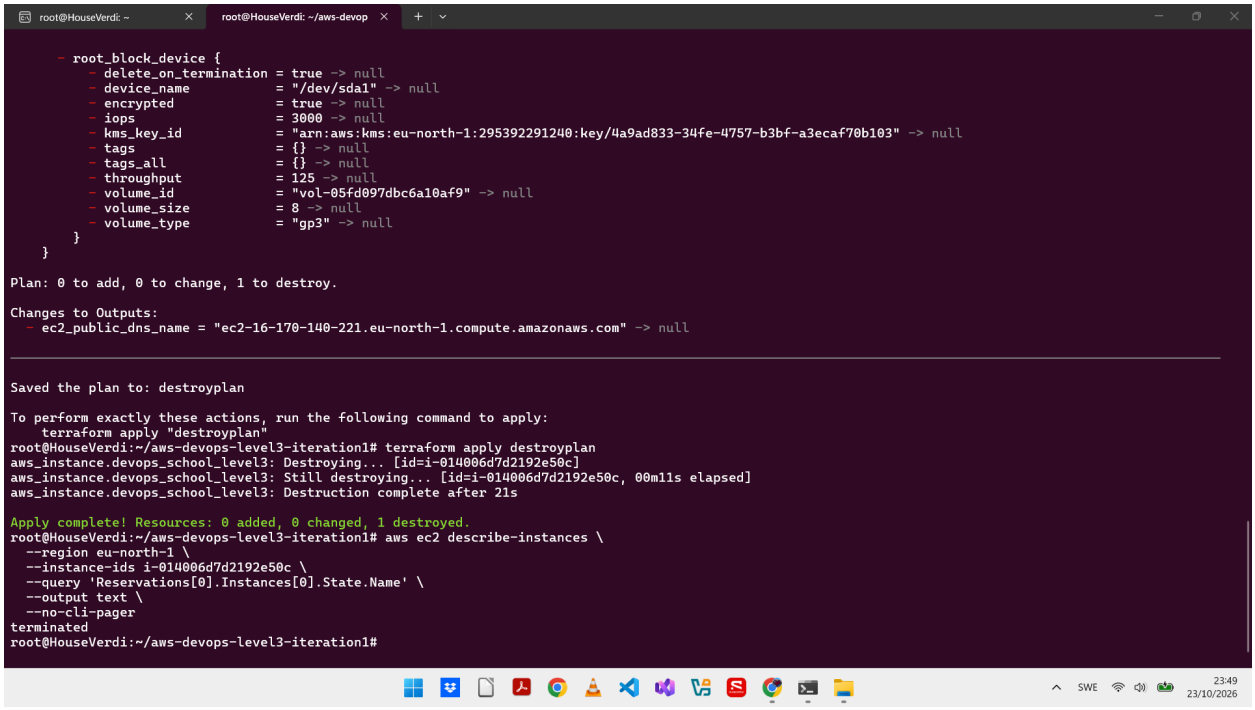


Bild 20. Slutförd borttagning och verifierad slutstatus terminated

```
terraform apply destroyplan
aws ec2 describe-instances --region eu-north-1 \
  --instance-ids i-014006d7d2192e50c \
  --query 'Reservations[0].Instances[0].State.Name' \
  --output text --no-cli-pager
```

Rötdisken var konfigurerad med delete_on_termination=true. Nyckelparet och säkerhetsgruppen skapades utanför Terraform och omfattades inte av denna borttagning. Webbplatsens tidigare IP och DNS dokumenterar testerna före avslutet.

Problem förbättringar och slutsats

Problem och lösningar

jq saknades lokalt och installerades. Ett upprepat AWS-kommando gav InvalidGroupId.Malformed eftersom SG_ID var tom i den aktuella terminalsessionen. Jag återställde värdet och kontrollerade att båda inkommande reglerna redan fanns.

Guidens filstruktur stämde inte med det klonade kodförrådet. Jag granskade filerna och publicerade heyme som index.html. Sidans faktiska text var Hey me! med litet m; webbtesterna dokumenterar den texten.

Förbättringar

Nästa steg kan vara att hantera säkerhetsgrupp och nyckelpar i Terraform, automatisera installation och publicering samt lägga till HTTPS. Nginx var enabled, men jag utförde inget omstartstest i denna iteration.

Slutsats och underlag

Arbetet visar skapande med Terraform, SSH-åtkomst, manuell publicering, fungerande HTTP-tester och verifierad borttagning. Terraformplanen kontrollerar infrastrukturen; Nginx och webbinnehållet verifierades separat.

Underlaget består av referensskriptet Level3 Iteration1 (1)(2)(1).sh, infogade skärmbilder från 23 oktober 2026 samt level3-1-terraform.txt, level3-1-terminal-history.txt och level3-1-server-history.txt. Den lokala terminalhistoriken innehåller även äldre kommandon; endast den aktuella sekvensen används här.

Bilaga Terraform filer

Filerna nedan återges från den exporterade konfigurationen. terraform.tfvars ska ligga separat från .tf-filerna.

providers tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables tf

```
variable "aws_region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "main_sg_id" {
  type = string
}

variable "ec2_type" {
  type = string
}

variable "ec2_name" {
  type = string
}

variable "keypair" {
  type = string
}
```

Bilaga instans och variabelvärden

main tf

```
resource "aws_instance" "devops_school_level3" {
  ami                         = var.ami_id
  instance_type               = var.ec2_type
  subnet_id                   = var.subnet_id
  key_name                    = var.keypair
  vpc_security_group_ids      = [var.main_sg_id]
  associate_public_ip_address = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = var.ec2_name
  }
}
```

outputs tf

```
output "ec2_public_dns_name" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.devops_school_level3.public_dns
}
```

terraform tfvars

```
aws_region = "eu-north-1"
ami_id     = "ami-06fb1802e2b757f09"
subnet_id  = "subnet-05d4dca4820d2eb94"
main_sg_id = "sg-0fb87d0f08a213466"
ec2_type   = "t3.micro"
ec2_name   = "devops-school-level3.tabish-admin"
keypair    = "devops.school.level3.tabish-admin"
```

Bilaga kommandon för servern

Följande kommandon finns i serverhistoriken och kördes efter SSH-inloggningen.

```
sudo apt-get update && \
  sudo DEBIAN_FRONTEND=noninteractive apt-get upgrade -y
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y nginx git
sudo systemctl enable --now nginx
systemctl is-active nginx
systemctl is-enabled nginx
git clone https://github.com/nenemustafa/Hellome.git \
  ~/hello-web && \
  ls -la ~/hello-web
sudo cp ~/hello-web/heyme /var/www/html/index.html && \
  sudo nginx -t && \
  sudo systemctl reload nginx
curl -i http://localhost/
exit
```

Lokala kommandon för Terraform och SSH

```
cd ~/aws-devops-level3-iteration1
terraform fmt
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
MY_EC2=$(terraform output -raw ec2_public_dns_name)
ssh -o StrictHostKeyChecking=accept-new \
  -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 "ubuntu@$MY_EC2"
```

Efter serverarbetet verifierades sidan från WSL och därefter kördes terraform plan, borttagningsplanen och slutkontrollen som dokumenteras tidigare i rapporten.